

Практическая работа №5

Теоретическое введение

Тестирование микросервисов является важным этапом разработки, поскольку микросервисная архитектура предполагает наличие множества отдельных, независимых сервисов, каждый из которых выполняет определенную функцию. Вот некоторые ключевые аспекты тестирования микросервисов:

Независимость сервисов: микросервисы должны быть независимыми друг от друга, что позволяет тестировать каждый сервис отдельно. Это означает, что изменения в одном сервисе не должны влиять на работу других сервисов.

Модульное тестирование: каждый микросервис должен иметь набор модульных тестов, которые тестируют его функциональность на уровне отдельных методов или функций. Модульное тестирование позволяет проверить корректность работы отдельных частей сервиса.

Интеграционное тестирование: поскольку микросервисы взаимодействуют друг с другом, необходимо также проводить интеграционное тестирование, чтобы убедиться, что они взаимодействуют правильно и корректно передают данные друг другу.

Тестирование API: Микросервисы обычно взаимодействуют друг с другом посредством API. Поэтому важно проводить тестирование API каждого сервиса для проверки корректности и надежности этого взаимодействия.

Тестирование производительности: помимо функционального тестирования, также важно проверить производительность микросервисов, особенно при высоких нагрузках. Это помогает обнаружить узкие места в архитектуре и оптимизировать их.

Тестирование отказоустойчивости: Микросервисы должны быть отказоустойчивыми, то есть способными обрабатывать сбои и ошибки без прерывания работы системы в целом. Поэтому тестирование отказоустойчивости также важно для микросервисной архитектуры.

Автоматизация тестирования: для обеспечения эффективного и надежного тестирования микросервисов рекомендуется автоматизировать тестовые процессы. Это позволяет быстро выполнять тесты и обнаруживать проблемы на ранних этапах разработки.

Тестирование микросервисов является неотъемлемой частью их разработки и поддержки, и его правильная организация помогает обеспечить надежность и стабильность работы всей системы.

Полезные ссылки

1. Официальная документация Docker: <https://docs.docker.com/>
2. Docker Hub: <https://hub.docker.com/>
3. Docker Community Forums: <https://forums.docker.com/>
4. Микросервисная архитектура: введение и основные концепции:
<https://microservices.io/>
5. Стратегии тестирования микросервисов:
<https://habr.com/ru/companies/serverspace/articles/690578/>

Задание

Цель работы: реализовать один из предложенных в теоретическом введении вариантов тестирования микросервисов

Данная практическая работа направлена на изучение вариантов тестирования, созданных ранее микросервисов. Тестирование позволяет выявлять ошибки, которые могут возникнуть при работе системы.

В ходе выполнения практики могут быть использованы любые языковые средства и фреймворки, для создания микросервисов.

Для примера будет использоваться модульное тестирование сервиса, приведенного в качестве примера в предыдущих практиках.

Пример файла test.py:

```
import unittest
from unittest.mock import patch
from task_service import app

class TestTaskService(unittest.TestCase):

    def test_create_task(self):
        # Мокаем метод отправки данных в БД или внешний
сервис
        with patch('task_service.create_task') as
mock_create_task:
            # Вызываем метод создания задачи
            app.create_task(description='Test task')

            # Проверяем, что метод был вызван с
правильными аргументами
```

```
mock_create_task.assert_called_once_with(description=
'Test task')
```

```
        # Добавим вывод сообщения об успешном
прохождении теста
```

```
        print("Unit test 'test_create_task' passed
successfully!")
```

```
if __name__ == '__main__':
    unittest.main()
```

Для автоматизации процесса тестирования, возможно его добавление в структуру Docker контейнера, чтобы при каждом его запуске выявить существующие ошибки.

Вопросы к практической работе

1. Что такое микросервисы и как они отличаются от монолитных приложений с точки зрения тестирования?
2. Какие виды тестирования микросервисов вы знаете?
3. Какие инструменты и библиотеки используются для автоматизации тестирования микросервисов?
4. Какие основные принципы и методы тестирования микросервисов существуют?
5. Какие особенности тестирования взаимодействия между микросервисами?
6. Что такое интеграционное тестирование микросервисов, и как оно проводится?
7. Как обеспечить изоляцию тестов микросервисов для поддержки независимости?
8. Каким образом проводится тестирование обновлений микросервисов в процессе развертывания?
9. Какие стратегии тестирования используются для обеспечения надежности и стабильности микросервисной архитектуры?
10. Как можно автоматизировать процесс тестирования микросервисов для повышения эффективности и скорости разработки?

Критерии оценки

- Реализовано тестирование разработанных микросервисов
- Проверена корректная работа тестов
- Составлен отчет о проделанной работе со скриншотами её выполнения
- Даны ответы на вопросы к практической работе